

Python E-Course

Module 7 — Smoothing & Filtering

Detailed Notes

Module Overview

Level: Filtering

Duration: ~1.5 weeks

Prerequisites: Modules 1-6

What You'll Learn

- Explain convolution with the sliding-window picture.
- Use box, Gaussian, median, and bilateral filters.
- Understand kernel size and sigma trade-offs.
- Write a pure-NumPy convolution.
- Pick the right filter for noise / smoothness / edge preservation.

Lessons

#	Lesson	Duration
1	Convolution & Correlation	35 min
2	Box / Mean Filter	25 min
3	Gaussian Filter	30 min
4	Median Filter	25 min
5	Bilateral Filter	30 min
6	Comparing Filters	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 8: Sharpening

Lesson 1: Convolution & Correlation

Module: 7 — Smoothing & Filtering

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Visualise convolution as a sliding window of multiply-and-sum.
- Distinguish convolution from correlation.
- Use `cv2.filter2D` for an arbitrary kernel.
- Write a pure-NumPy convolution to demystify what libraries do.

Prerequisites

- Module 1, [reference/math-refresher.md §3](#)

1. Why This Matters

Every linear filter (blur, sharpen, edge) is a convolution with a different kernel. Once you can picture the sliding window, every filter in this module is the same operation with a different small grid of numbers.

2. Concept

2.1 The sliding window picture

For a 3×3 kernel and a small image patch:

Image patch:		Kernel:		Output pixel at the patch centre:
$\begin{bmatrix} p_0 & p_1 & p_2 \\ p_3 & p_4 & p_5 \\ p_6 & p_7 & p_8 \end{bmatrix}$	*	$\begin{bmatrix} k_0 & k_1 & k_2 \\ k_3 & k_4 & k_5 \\ k_6 & k_7 & k_8 \end{bmatrix}$	=	$p_0 \cdot k_0 + p_1 \cdot k_1 + p_2 \cdot k_2 +$ $p_3 \cdot k_3 + p_4 \cdot k_4 + p_5 \cdot k_5 +$ $p_6 \cdot k_6 + p_7 \cdot k_7 + p_8 \cdot k_8$

Slide the kernel one pixel at a time over the image. At each location, multiply element-wise and sum. The result is one output pixel.

2.2 ASCII walkthrough

For 1D simplicity:

```
input :   5 10 15 20 25 30 35 40
kernel:  [1 2 1]   (sum 4 → divide by 4 for average)

position 0 (centred on 10):
  5*1 + 10*2 + 15*1 = 40   /4 = 10

position 1 (centred on 15):
  10*1 + 15*2 + 20*1 = 60  /4 = 15
```

```
position 2 (centred on 20):  
15*1 + 20*2 + 25*1 = 80 /4 = 20
```

The kernel $[1, 2, 1] / 4$ is the simplest weighted blur — it picks up the centre value most strongly.

2.3 Convolution vs Correlation

	Operation	Kernel handling
Correlation	multiply-sum	as-is
Convolution	multiply-sum	flip both axes first

For symmetric kernels (most blurs), they're identical. For directional kernels (Sobel), they differ. OpenCV's `cv2.filter2D` is correlation by default — flip your kernel yourself if you want strict convolution.

2.4 Borders

What happens at the edge, where the kernel hangs off the image? Same `borderType` options as `cv2.copyMakeBorder`:

- `cv2.BORDER_REFLECT_101` (default for most filters) — mirror
- `BORDER_CONSTANT` — pad with 0
- `BORDER_REPLICATE` — repeat edge
- `BORDER_WRAP` — tile

2.5 cv2.filter2D

```
out = cv2.filter2D(img, ddepth=-1, kernel=K)
```

- `ddepth=-1` keeps the input's dtype. Use `cv2.CV_32F` if you need float output (for kernels with negative values).
- `kernel` is a NumPy array, typically float32, summing to 1.0 for true averaging.

2.6 Pure-NumPy convolution (educational)

```
import numpy as np  
  
def convolve2d(img, k):  
    kh, kw = k.shape  
    pad_h, pad_w = kh // 2, kw // 2  
    padded = np.pad(img, ((pad_h, pad_h), (pad_w, pad_w)), mode='reflect')  
    out = np.zeros_like(img, dtype=np.float32)  
    for y in range(img.shape[0]):  
        for x in range(img.shape[1]):  
            patch = padded[y:y+kh, x:x+kw]  
            out[y, x] = (patch * k).sum()  
    return np.clip(out, 0, 255).astype(np.uint8)
```

Slow but illustrative. OpenCV's C-level implementation is hundreds of times faster.

3. Code Examples

Example 1: filter2D with a custom kernel

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
# Simple 3x3 averaging kernel
k = np.ones((3, 3), np.float32) / 9
out = cv2.filter2D(img, -1, k)
cv2.imwrite("box3.png", out)
print("input range:", img.min(), img.max())
print("output range:", out.min(), out.max())
```

Expected Output: A slightly blurred cameraman. Console confirms output stays in 0-255.

Example 2: Pure-NumPy convolve, verify against OpenCV

```
import cv2, numpy as np
from skimage.data import camera

def convolve2d(img, k):
    kh, kw = k.shape
    pad_h, pad_w = kh // 2, kw // 2
    padded = np.pad(img, ((pad_h, pad_h), (pad_w, pad_w)), mode='reflect')
    out = np.zeros_like(img, dtype=np.float32)
    for y in range(img.shape[0]):
        for x in range(img.shape[1]):
            out[y, x] = (padded[y:y+kh, x:x+kw] * k).sum()
    return np.clip(out, 0, 255).astype(np.uint8)

img = camera()[:100, :100] # tiny patch (slow loop)
k = np.ones((3, 3), np.float32) / 9
mine = convolve2d(img, k)
cv2_out = cv2.filter2D(img, -1, k)

print("max abs diff:", np.abs(mine.astype(int) - cv2_out.astype(int)).max())
```

Expected Output: max abs diff: 0 or 1 — tiny rounding difference, otherwise identical.

Example 3: Directional kernel

```
import cv2, numpy as np
from skimage.data import coins

img = coins().astype(np.float32)
sobel_x = np.array([[ -1,  0,  1],
                    [ -2,  0,  2],
                    [ -1,  0,  1]], dtype=np.float32)
out = cv2.filter2D(img, cv2.CV_32F, sobel_x) # float output (has negatives)
out_vis = np.abs(out).clip(0, 255).astype(np.uint8)
cv2.imwrite("sobel_x.png", out_vis)
```

Expected Output: A grayscale image where vertical edges (the coin edges, mostly vertical) are bright. Horizontal edges are dim. (Fully covered in M9.)

Example 4: Border behaviour

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
k = np.ones((11, 11), np.float32) / 121

for border, name in [(cv2.BORDER_REFLECT_101, "reflect101"),
                    (cv2.BORDER_REPLICATE, "replicate"),
                    (cv2.BORDER_CONSTANT, "constant")]:
    out = cv2.filter2D(img, -1, k, borderType=border)
    cv2.imwrite(f"blur_{name}.png", out)
```

Expected Output: Three blurred versions. constant (zero-pad) darkens the edge of the result; replicate and reflect look natural.

4. Parameter Sensitivity

Param	Effect
Kernel sum	=1: brightness preserved. >1: brighter. <1: darker.
Kernel size	Bigger = wider influence per output pixel
ddepth=-1	Output dtype = input

5. Common Mistakes

Mistake	What Happens	Fix
Kernel doesn't sum to 1	Brightness shifts	Normalize: $k / k.sum()$
Use uint8 ddepth for negative kernel	Wraps to garbage	Use cv2.CV_32F, then np.abs if needed
Forget that filter2D is correlation	Wrong direction for Sobel	Flip kernel for strict convolution
Wrong border default	Edges look weird	Stick with BORDER_REFLECT_101

6. Practice Tasks

- Build a 5x5 averaging kernel and apply it to camera().
- Build a 3x3 sharpening kernel: $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$. Apply, save.
- Implement the pure-NumPy convolve2d and verify it matches OpenCV on a small patch.

7. Key Takeaways

- Convolution = sliding-window multiply-sum.
- Convolution vs correlation: flip the kernel (matters only for asymmetric kernels).
- `cv2.filter2D(img, -1, kernel)` is the workhorse.
- Negative kernel values → use CV_32F output then handle the sign.

8. What's Next

The simplest blur — box / mean filter.

Lesson 2: Box / Mean Filter

Module: 7 — Smoothing & Filtering

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Apply `cv2.blur` and `cv2.boxFilter`.
- Predict the effect of kernel size.
- Recognize why box blur is rarely the best choice.

Prerequisites

- Module 7 Lesson 1

1. Why This Matters

The simplest blur — uniform average over a window. Cheap, easy to reason about, but visually it produces blocky "boxy" softening. It's a useful baseline before reaching for Gaussian.

2. Concept

2.1 The kernel

For a (k, k) box filter, every entry is $1/(k*k)$. A 3×3 box:

```
[ 1/9  1/9  1/9 ]
[ 1/9  1/9  1/9 ]
[ 1/9  1/9  1/9 ]
```

Each output pixel is the average of the $k*k$ pixels around it.

2.2 OpenCV functions

```
cv2.blur(img, (k, k))                # normalised box (most common)
cv2.boxFilter(img, -1, (k, k), normalize=True)  # equivalent
cv2.boxFilter(img, -1, (k, k), normalize=False) # SUM (not average)
```

`cv2.blur` is the same as `cv2.boxFilter` with `normalize=True`.

2.3 Kernel size

Bigger $k \rightarrow$ more blur, but also slower ($O(k^2)$ per pixel). For very large blurs use Gaussian (separable, faster) or repeat a small box blur.

2.4 Why "boxy"

The kernel has hard edges — pixels just inside the window contribute as much as pixels at the centre. Result: blurring is not smooth. Diagonal lines look stepped.

3. Code Examples

Example 1: Box blur, multiple sizes

```
import cv2, numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
for k in [3, 7, 15, 31]:
    out = cv2.blur(img, (k, k))
    cv2.imwrite(f"box_k{k}.png", out)
```

Expected Output: Four versions. $k=3$ is barely blurred. $k=31$ looks heavily smoothed; fine detail (hair, helmet text) is gone.

Example 2: blur vs boxFilter

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
a = cv2.blur(img, (5, 5))
b = cv2.boxFilter(img, -1, (5, 5), normalize=True)
print("equal:", np.array_equal(a, b))
```

Expected Output: equal: True

Example 3: Box vs Gaussian (preview of next lesson)

```
import cv2
from skimage.data import camera

img = camera()
box = cv2.blur(img, (15, 15))
gaus = cv2.GaussianBlur(img, (15, 15), 0)

cv2.imwrite("box_15.png", box)
cv2.imwrite("gaus_15.png", gaus)
```

Expected Output: Both blurred but Gaussian looks smoother. Box has a slightly "blocky" quality on diagonal edges.

4. Parameter Sensitivity

Kernel size	Effect
3×3	barely visible smoothing
7×7	clear softening of small detail
15×15	heavy blur; text becomes unreadable
31+	extreme — usually better off with Gaussian σ tuning

5. Common Mistakes

Mistake	What Happens	Fix
---------	--------------	-----

Even-sized kernel	OpenCV anchor offset; works but check	Prefer odd sizes
normalize=False accidentally	Output is SUM, overflows uint8	Use cv2.blur for averaging
Box for "natural" blur	Looks boxy	Use Gaussian

6. Practice Tasks

- Apply box blur sizes 3, 7, 15, 31 to camera(). Compare.
- Verify `cv2.blur == cv2.boxFilter(..., normalize=True)`.
- Time `cv2.blur((31,31))` vs `cv2.GaussianBlur((31,31))` on a 1MP image.

7. Key Takeaways

- Box filter = uniform average.
- `cv2.blur(img, (k, k))` is the standard call.
- Odd kernel sizes; bigger = more blur, slower.
- Less natural-looking than Gaussian — use Gaussian when in doubt.

8. What's Next

Gaussian blur — the smooth, natural-looking default.

Lesson 3: Gaussian Filter

Module: 7 — Smoothing & Filtering

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply `cv2.GaussianBlur`.
- Tune kernel size and sigma σ .
- Know that Gaussian is separable ($1D \times 1D$), which makes it fast.

Prerequisites

- Module 7 Lesson 2

1. Why This Matters

Gaussian blur is the default blur for almost everything: pre-smoothing for edge detection (M9), noise reduction (M15), preview thumbnails. It produces visually pleasing, isotropic softening with no boxy artefacts.

2. Concept

2.1 The Gaussian function

In 2D:

$$G(x, y) = (1 / (2\pi\sigma^2)) \cdot \exp(- (x^2 + y^2) / (2\sigma^2))$$

- σ (sigma) controls the width of the bell curve. Bigger σ = wider, smoother blur.
- The kernel is normalised so it sums to 1.

2.2 The kernel visualisation

A 5×5 Gaussian kernel with $\sigma = 1$ looks roughly:

```
[0.003 0.013 0.022 0.013 0.003]
[0.013 0.060 0.098 0.060 0.013]
[0.022 0.098 0.162 0.098 0.022]
[0.013 0.060 0.098 0.060 0.013]
[0.003 0.013 0.022 0.013 0.003]
```

Heavier weight at the centre, falling off smoothly to the edges — that's why it doesn't look boxy.

2.3 OpenCV API

```
out = cv2.GaussianBlur(img, (k, k), sigmaX)
```

- (k, k) kernel size — must be odd.
- `sigmaX = 0` lets OpenCV pick σ from the kernel size: $\sigma = 0.3 * ((k-1)/2 - 1) + 0.8$.
- You can also call `cv2.GaussianBlur(img, (0, 0), sigma)` to let σ drive size.

2.4 Separability — why it's fast

Gaussian is separable: a 2D Gaussian blur is mathematically equivalent to a 1D horizontal Gaussian blur followed by a 1D vertical Gaussian blur. That's $O(2k)$ per pixel instead of $O(k^2)$.

OpenCV does this automatically. To see it explicitly:

```
out = cv2.sepFilter2D(img, -1, kx, ky) # general separable filter
```

2.5 Picking σ

- Small (0.5–1.5) → mild blur, preserves edges decently.
- Medium (2–4) → noise reduction without losing structure.
- Large (5+) → strong smoothing, edges lost.

3. Code Examples

Example 1: Tune sigma

```
import cv2
from skimage.data import camera

img = camera()
for sigma in [0.5, 1.5, 3.0, 6.0]:
    out = cv2.GaussianBlur(img, (0, 0), sigma)
    cv2.imwrite(f"gauss_{sigma}.png", out)
```

Expected Output: Four versions — $\sigma=0.5$ nearly unchanged; $\sigma=6$ heavily smoothed (cameraman becomes blurry).

Example 2: Build the kernel explicitly

```
import cv2, numpy as np

k = 5
sigma = 1.0
kx = cv2.getGaussianKernel(k, sigma) # column (k, 1)
ky = cv2.getGaussianKernel(k, sigma)
g2d = kx @ ky.T # (k, k)
print("kernel sum:", g2d.sum()) # ~1.0
print(g2d.round(3))
```

Expected Output: A 5×5 matrix summing to ~1.0, with the peak at the centre.

Example 3: Verify separability

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
direct = cv2.GaussianBlur(img, (7, 7), 1.5)

k = cv2.getGaussianKernel(7, 1.5)
sep = cv2.sepFilter2D(img, -1, k, k)
```

```
print("max abs diff:", np.abs(direct.astype(int) - sep.astype(int)).max())
```

Expected Output: max abs diff: 0 (or ≤ 1 due to rounding). Same result, both paths.

Example 4: Speed comparison vs box on big kernel

```
import cv2, time
import numpy as np
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
big = cv2.resize(img, (2048, 2048))

for name, fn in [("box", lambda: cv2.blur(big, (31, 31))),
                 ("gaussian", lambda: cv2.GaussianBlur(big, (31, 31), 0))]:
    t0 = time.perf_counter()
    fn()
    print(f"{name}: {(time.perf_counter() - t0) * 1000:.1f} ms")
```

Expected Output: Both are fast at this kernel size (a few ms each). For much larger kernels (101+) the separability advantage of Gaussian shows more clearly.

4. Parameter Sensitivity

σ	Effect
0.5	Subtle anti-aliasing
1–2	Good for noise reduction
3–4	Heavy smoothing — pre-blur for edge detection
6+	Extreme; almost destroys structure
Kernel size	Effect
Smaller than $\sim 6\sigma$	Truncates the Gaussian; less smooth
$\geq 6\sigma$	Captures essentially all of the Gaussian

Rule of thumb: $k \approx 6\sigma + 1$ (rounded to odd).

5. Common Mistakes

Mistake	What Happens	Fix
Even kernel size	Error	Use odd
sigma = 0 but tiny kernel	OpenCV picks tiny $\sigma \rightarrow$ almost no blur	Pass a sane σ explicitly
Huge kernel, σ too small	Wasted work; same as small kernel	Match $k \approx 6\sigma$
Use box where Gaussian would be cleaner	Boxy artefacts	Use Gaussian by default

6. Practice Tasks

- Apply $\sigma \in \{0.5, 1.5, 3, 6\}$ on camera(); visually compare.
- Build a 7×7 Gaussian kernel from cv2.getGaussianKernel.

- Verify separable form gives the same output as direct GaussianBlur.

7. Key Takeaways

- Gaussian = smooth bell-curve weights, σ controls width.
- `cv2.GaussianBlur(img, (k, k),  $\sigma$ )`.
- Separable: 2D is two 1D passes — fast.
- $k \approx 6\sigma + 1$ as a sizing rule.

8. What's Next

Median filter — non-linear, great for salt-and-pepper noise.

Lesson 4: Median Filter

Module: 7 — Smoothing & Filtering

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Apply `cv2.medianBlur`.
- Understand why median preserves edges while removing salt-and-pepper.
- Pick kernel size for impulse-noise removal.

Prerequisites

- Module 7 Lesson 3

1. Why This Matters

Salt-and-pepper noise (random pure-white and pure-black pixels) destroys Gaussian / box blur output — the noise is averaged in. Median filter removes it outright while keeping edges sharp. It's the go-to for impulse noise.

2. Concept

2.1 The idea — non-linear

For each pixel, sort the window's pixel values and take the median. Outliers (the salt and pepper) sit at the extremes and never get picked.

```
window: [10, 12, 15, 255, 11, 13, 14, 9, 16]
sorted: [9, 10, 11, 12, 13, 14, 15, 16, 255]
median: 13      <- the outlier (255) is ignored
```

2.2 OpenCV API

```
out = cv2.medianBlur(img, ksize)
```

`ksize` is the edge length of the square window. Must be odd. For `ksize > 5`, OpenCV uses an optimized histogram-based algorithm.

2.3 Edge preservation

Average-based filters (box, Gaussian) blur edges because they include both sides of an edge in the average. Median picks one side or the other — it doesn't smear.

2.4 Where it fails

- Heavy Gaussian noise (continuous, not impulse) — median doesn't help much.
- Very fine textures — median can wipe them out.
- Large kernel sizes (>9) — gets slow and can over-smooth.

2.5 Comparison context

Filter	Best at	Hurts
Box	Quick blur	Edges, looks boxy
Gaussian	Smooth blur, Gaussian noise	Edges (some)
Median	Salt-and-pepper / impulse noise	Fine textures
Bilateral	Smooth, edge-preserving denoising	Slow

3. Code Examples

Example 1: Add salt-and-pepper, then median

```
import cv2, numpy as np
from skimage.data import camera

img = camera().copy()
noisy = img.copy()
np.random.seed(0)
# 5% white spots, 5% black spots
mask = np.random.rand(*img.shape)
noisy[mask < 0.05] = 0
noisy[mask > 0.95] = 255

cv2.imwrite("cam_noisy.png", noisy)
for k in [3, 5, 7]:
    out = cv2.medianBlur(noisy, k)
    cv2.imwrite(f"cam_median_k{k}.png", out)
```

Expected Output: cam_noisy.png is dotted with pure-white and pure-black specks. cam_median_k3.png removes most specks while keeping the cameraman's outline sharp. k=5/7 removes all noise but slightly soften textures.

Example 2: Median vs Gaussian on noisy image

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = img.copy()
mask = np.random.rand(*img.shape)
noisy[mask < 0.05] = 0
noisy[mask > 0.95] = 255

med = cv2.medianBlur(noisy, 3)
gau = cv2.GaussianBlur(noisy, (3, 3), 0)

cv2.imwrite("compare_median.png", med)
cv2.imwrite("compare_gaussian.png", gau)
```


Expected Output: Median: noise gone, edges still sharp. Gaussian: noise smeared into blurry blobs — visibly worse.

Example 3: Larger kernel on heavier noise

```
import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
noisy = img.copy()
mask = np.random.rand(*img.shape)
noisy[mask < 0.15] = 0
noisy[mask > 0.85] = 255

for k in [3, 5, 7, 9]:
    cv2.imwrite(f"heavy_k{k}.png", cv2.medianBlur(noisy, k))
```

Expected Output: k=3 leaves a fair amount of speckle (since 30% of pixels are noisy, sometimes the median window has more bad than good); k=5–7 cleans it; k=9 starts to soften texture.

4. Parameter Sensitivity

ksize	Effect
3	Removes light impulse noise; preserves edges well
5	Stronger; handles ~10% noise
7	Heavy; handles ~20% noise; some texture loss
9+	Slow; over-smooth; rarely needed

5. Common Mistakes

Mistake	What Happens	Fix
Even ksize	Error	Use odd
Median on color directly	Per-channel median; OK but slight color shift possible	OK in practice; convert to LAB if perfectionist
Median to fix Gaussian noise	Doesn't help much	Use Gaussian / NLM / bilateral (M15)
Massive ksize	Slow + texture loss	Try ksize 3 or 5 first

6. Practice Tasks

- Add 5% salt-and-pepper to camera(). Filter with median(3) and Gaussian(3,3). Compare.
- Try ksize 3, 5, 7 on 10% noise. Find the smallest that cleans it.
- Apply median to a color image (e.g. coffee()). Check for color shifts.

7. Key Takeaways

- Median = pick the middle value of the window.
- Best at salt-and-pepper / impulse noise.

- Preserves edges (unlike averaging filters).
- ksize odd; 3 or 5 is usually enough.

8. What's Next

Bilateral filter — the smart one that combines smoothing with edge preservation for any noise type.

Lesson 5: Bilateral Filter

Module: 7 — Smoothing & Filtering

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply `cv2.bilateralFilter`.
- Explain why bilateral preserves edges (the range kernel).
- Tune `d`, `sigmaColor`, `sigmaSpace`.

Prerequisites

- Module 7 Lesson 4

1. Why This Matters

Gaussian smooths but blurs edges. Median preserves edges but only handles impulse noise. Bilateral does both: smooth flat regions while keeping edges crisp. It's the magic filter for portrait skin-smoothing, denoising photos before processing, and cartoon-style effects.

2. Concept

2.1 Two kernels at once

A regular Gaussian weights pixels by spatial distance. The bilateral filter also weights by intensity distance:

$$\text{weight} = G_{\text{spatial}}(\text{dx}, \text{dy}) \cdot G_{\text{range}}(I_{\text{neighbour}} - I_{\text{centre}})$$

If a neighbour pixel is far in intensity (across an edge), its range weight is near zero — it doesn't influence the output. Result: edges are preserved.

2.2 OpenCV API

```
out = cv2.bilateralFilter(img, d, sigmaColor, sigmaSpace)
```

Param	Meaning
d	Diameter of the spatial neighbourhood. 0 or -1 = computed from sigmaSpace. Typical 5-9 for fast, 15+ for heavy.
sigmaColor	σ of the range Gaussian — how dissimilar (in intensity) a neighbour can be and still count. Bigger = more aggressive smoothing across edges.
sigmaSpace	σ of the spatial Gaussian — how wide the neighbourhood.

2.3 Pick params

- Light smoothing: d=5, sigmaColor=25, sigmaSpace=25
- Heavy "skin-smooth" effect: d=9, sigmaColor=75, sigmaSpace=75
- Even heavier / cartoon-like: d=15, sigmaColor=150, sigmaSpace=150

2.4 Cost

Bilateral is slow — non-separable, expensive math. For previews / 1-MP images it's fine; for video at 1080p you'll need cv2.adaptiveBilateralFilter (not available in all builds) or a downsample-then-bilateral-then-upsample trick.

2.5 Color images

Works directly on BGR — each channel filtered independently in OpenCV's implementation, but using a joint range kernel computed in some color metric. Result preserves color edges well.

3. Code Examples

Example 1: Bilateral vs Gaussian

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
gaus = cv2.GaussianBlur(img, (9, 9), 5)
bilat = cv2.bilateralFilter(img, 9, 75, 75)
cv2.imwrite("astro_gauss.png", gaus)
cv2.imwrite("astro_bilat.png", bilat)
```

Expected Output: astro_gauss.png softens everything including edges (face features blurry). astro_bilat.png smooths smooth regions (cheeks, helmet plastic) but keeps eyes, mouth, helmet outline crisp.

Example 2: Cartoon-like effect (heavy bilateral + edges)

```
import cv2
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
b = cv2.bilateralFilter(img, 15, 150, 150)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
                             cv2.THRESH_BINARY, 9, 8)
edges_bgr = cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR)
cartoon = cv2.bitwise_and(b, edges_bgr)
cv2.imwrite("cartoon.png", cartoon)
```

Expected Output: Astronaut as a cartoon — flat color regions (heavy bilateral smoothing) with strong black outlines (adaptive threshold edges).

Example 3: Tune sigmaColor

```
import cv2
from skimage.data import coffee
```

```
img = cv2.cvtColor(coffee(), cv2.COLOR_RGB2BGR)
for sc in [25, 75, 150]:
    out = cv2.bilateralFilter(img, 9, sc, 50)
    cv2.imwrite(f"coffee_sc{sc}.png", out)
```

Expected Output: Three versions. sc=25 barely smooths across small intensity differences (preserves nearly all detail). sc=150 smooths across larger gradients (looks more painterly).

Example 4: Speed timing

```
import cv2, time
from skimage.data import astronaut

img = cv2.cvtColor(astronaut(), cv2.COLOR_RGB2BGR)
img = cv2.resize(img, (1024, 1024))

t = time.perf_counter()
cv2.bilateralFilter(img, 9, 75, 75)
print(f"bilateral d=9 : {(time.perf_counter()-t)*1000:.1f} ms")

t = time.perf_counter()
cv2.bilateralFilter(img, 15, 150, 150)
print(f"bilateral d=15 : {(time.perf_counter()-t)*1000:.1f} ms")

t = time.perf_counter()
cv2.GaussianBlur(img, (15, 15), 0)
print(f"gaussian k=15 : {(time.perf_counter()-t)*1000:.1f} ms")
```

Expected Output (timings vary):

```
bilateral d=9 : 280.0 ms
bilateral d=15 : 720.0 ms
gaussian k=15 : 4.5 ms
```

4. Parameter Sensitivity

Param	Lower	Higher
d	faster, smaller neighbourhood	slower, broader smoothing
sigmaColor	less smoothing across intensity steps (more edges preserved)	smooths across bigger steps (less edge preservation)
sigmaSpace	small region	wider region

Rule of thumb: keep sigmaColor and sigmaSpace roughly equal.

5. Common Mistakes

Mistake	What Happens	Fix
Use on large images at full res in a loop	Painfully slow	Downsample first
sigmaColor too large	Acts like Gaussian (loses edge preservation)	Try 25-75

d very large with small sigmas	Wasted work	Match $d \approx 6 * \text{sigmaSpace}$
Use bilateral as the only denoiser	Misses impulse noise	Combine with median

6. Practice Tasks

- Apply bilateral and Gaussian with similar smoothing strength on astronaut(). Compare edge preservation.
- Build a cartoon effect by combining heavy bilateral + adaptive threshold edges.
- Time bilateral vs Gaussian on a 1MP image — note the order-of-magnitude difference.

7. Key Takeaways

- Bilateral = spatial Gaussian $\times$ range Gaussian.
- Preserves edges while smoothing flat regions.
- Slower than Gaussian by 50-100 $\times$.
- Sweet spot params: $d=9$, $\text{sigmaColor}=75$, $\text{sigmaSpace}=75$ for moderate skin-smooth.

8. What's Next

Compare all four filters side by side.

Lesson 6: Comparing Filters

Module: 7 — Smoothing & Filtering

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Pick the right filter for the job.
- Build a side-by-side comparison rig.
- Use the comparison rig to debug noise problems.

Prerequisites

- All of Module 7 lessons 2-5.

1. Why This Matters

Knowing the four filters individually is one thing. Knowing which to reach for when you see a particular noise pattern is the practical skill. This lesson is a decision tree + a comparison harness you can keep.

2. Concept

2.1 Decision tree

Is the noise mostly random white/black specks (salt-and-pepper)?

YES -> medianBlur (k=3 or 5)

NO -> Is it grainy / Gaussian-style?

YES -> Need edges sharp?

YES -> bilateralFilter (or NLN, M15)

NO -> GaussianBlur (sigma ~ 1-2)

NO -> Just want a soft preview?

YES -> GaussianBlur or blur

2.2 Summary table

Filter	Cost	Removes	Preserves edges?	Good for
blur (box)	low	mild noise	no	quick preview
GaussianBlur	low	mild Gaussian noise	no	denoise / pre-edge
medianBlur	medium	salt-and-pepper	yes	impulse noise
bilateralFilter	high	heavy noise / smoothing	yes	portraits / cartoons

2.3 The comparison rig

A 5-panel display: original + 4 filters. Helps you eyeball which is best:

```
def compare(img, k=5, sigma=1.5, sigmaColor=75, sigmaSpace=75):  
    box = cv2.blur(img, (k, k))
```

```

    gaus = cv2.GaussianBlur(img, (0, 0), sigma)
    med = cv2.medianBlur(img, k if k % 2 else k + 1)
    bilt = cv2.bilateralFilter(img, 9, sigmaColor, sigmaSpace)
    return np.hstack([img, box, gaus, med, bilt])

```

2.4 Synthetic noise for testing

```

def add_gaussian_noise(img, sigma=20):
    n = np.random.normal(0, sigma, img.shape).astype(np.int16)
    return np.clip(img.astype(np.int16) + n, 0, 255).astype(np.uint8)

def add_salt_pepper(img, p=0.05):
    out = img.copy()
    m = np.random.rand(*img.shape[:2])
    out[m < p] = 0
    out[m > 1 - p] = 255
    return out

```

Use these to demonstrate which filter wins on which noise.

3. Code Examples

Example 1: Compare on Gaussian noise

```

import cv2, numpy as np
from skimage.data import camera

def add_gaussian_noise(img, sigma=20):
    n = np.random.normal(0, sigma, img.shape).astype(np.int16)
    return np.clip(img.astype(np.int16) + n, 0, 255).astype(np.uint8)

img = camera()
noisy = add_gaussian_noise(img, sigma=25)

box = cv2.blur(noisy, (5, 5))
gaus = cv2.GaussianBlur(noisy, (0, 0), 1.5)
med = cv2.medianBlur(noisy, 5)
bilt = cv2.bilateralFilter(noisy, 9, 75, 75)

combo = np.hstack([noisy, box, gaus, med, bilt])
cv2.imwrite("compare_gaussian_noise.png", combo)

```

Expected Output: 5-panel image: noisy, then 4 cleaned versions. Gaussian and bilateral handle Gaussian noise well (bilateral preserves edges better); median is mediocre on this kind of noise.

Example 2: Compare on salt-and-pepper

```

import cv2, numpy as np
from skimage.data import camera

img = camera()
np.random.seed(0)
m = np.random.rand(*img.shape)
sp = img.copy()
sp[m < 0.05] = 0; sp[m > 0.95] = 255

```



```

box = cv2.blur(sp, (5, 5))
gaus = cv2.GaussianBlur(sp, (0, 0), 1.5)
med = cv2.medianBlur(sp, 3)
bilt = cv2.bilateralFilter(sp, 9, 75, 75)

```

```

combo = np.hstack([sp, box, gaus, med, bilt])
cv2.imwrite("compare_salt_pepper.png", combo)

```

Expected Output: Same 5-panel. Median wins clearly — the others smear the noise into blurry blobs.

Example 3: Print which params for which task

```

recipes = {
    "preview blur": ("blur", {"ksize": (15, 15)}),
    "pre-edge smoothing": ("Gaussian", {"sigma": 1.4}),
    "salt-and-pepper noise": ("median", {"ksize": 3}),
    "portrait skin smoothing": ("bilateral", {"d": 9, "sigmaColor": 75, "sigmaSpace":
75}),
    "cartoon effect": ("bilateral", {"d": 15, "sigmaColor": 150,
"sigmaSpace": 150}),
}
for k, v in recipes.items():
    print(f"{k:30s} -> {v[0]:10s} {v[1]}")

```

Expected Output: A neat printed table of which filter and params to reach for in common situations.

4. Parameter Sensitivity

(Per-filter — see Lessons 2-5.)

5. Common Mistakes

Mistake	What Happens	Fix
Pick the most expensive filter by default	Slow	Start with Gaussian; escalate if needed
Use one filter for every noise	Sub-optimal	Match filter to noise type
Wrong ksize parity	Error or odd behaviour	Always odd

6. Practice Tasks

- Build a compare function and run on camera() + Gaussian noise.
- Build a compare function and run on camera() + salt-and-pepper.
- Add timing per filter; print them in the comparison.

7. Key Takeaways

- Decision tree: noise type → filter.
- Median for impulse; Gaussian for mild Gaussian; bilateral for heavy + edge-aware.
- Comparison rig is the fastest debugging tool when you're unsure.

8. What's Next

End of Module 7. Next: Module 8 — Sharpening — the opposite operation.

Module 7 — Exercises

18 exercises.

7.1 Convolution

- (E) Apply a 5×5 averaging kernel with `cv2.filter2D` to `camera()`.
- (M) Apply the 3×3 sharpen kernel `[[0,-1,0],[-1,5,-1],[0,-1,0]]`.
- (H) Implement pure-NumPy `convolve2d`; verify against OpenCV on a small patch.

7.2 Box

- (E) Apply `cv2.blur((3,3))` and `(15,15)` to `camera()`; compare.
- (M) Time `cv2.blur((31,31))` on a 2000×2000 image.
- (H) Compare `box(15,15)` vs `Gaussian(15,15)` visually on `astronaut()`.

7.3 Gaussian

- (E) Apply $\sigma \in \{0.5, 2, 5\}$ to `camera()`.
- (M) Build a 7×7 Gaussian kernel from `cv2.getGaussianKernel` and apply via `sepFilter2D`.
- (H) Verify separable form == direct `GaussianBlur`.

7.4 Median

- (E) Add 5% salt-pepper; filter with `median(3)`.
- (M) Compare `median(3)` vs `Gaussian(3,3)` on salt-pepper noise.
- (H) Find smallest `ksize` that cleans 15% noise.

7.5 Bilateral

- (E) Apply `bilateralFilter(d=9, sigmaColor=75, sigmaSpace=75)` to `astronaut()`.
- (M) Tune `sigmaColor` $\in \{25, 75, 150\}$.
- (H) Build a cartoon effect: heavy bilateral + adaptive threshold edges.

7.6 Compare

- (E) Add Gaussian noise; run 4-filter compare rig.
- (M) Add salt-pepper; do the same.
- (H) Time each filter; print "noise type recommendation" table.

Module 7 — Quiz

10 MCQs.

Q1

Convolution differs from correlation by: A. flipping the kernel B. summing C. dot product D. transpose

Answer: A (L1)

Q2

Gaussian blur is fast because it's: A. cached B. separable ($1D \times 1D$) C. in C++ D. uses GPU

Answer: B (L3)

Q3

For salt-and-pepper noise, the right filter is: A. box B. Gaussian C. median D. bilateral

Answer: C (L4)

Q4

For smooth edge-preserving denoising, use: A. box B. Gaussian C. median D. bilateral

Answer: D (L5)

Q5

`cv2.blur(img, (7,7))` is the same as: A. `cv2.GaussianBlur` with $\sigma=7$ B. `cv2.boxFilter(... normalize=True)` C. `cv2.medianBlur` D. `cv2.filter2D` with sharpening

Answer: B (L2)

Q6

The rule of thumb relating Gaussian kernel size to σ : A. $k = \sigma$ B. $k = 3\sigma$ C. $k \approx 6\sigma + 1$ (odd) D. $k = 100$

Answer: C (L3)

Q7

Bilateral params: roughly which orders of magnitude? A. $d=1$, $\sigma=1$ B. $d=9$, $\sigma=75$ C. $d=100$, $\sigma=1000$ D. $d=0$, $\sigma=0$

Answer: B (L5)

Q8

`cv2.medianBlur` ksize must be: A. even B. odd C. multiple of 4 D. ≥ 7

Answer: B (L4)

Q9

A 3×3 kernel of all $1/9$ produces: A. sharpening B. edge detection C. box blur D. negative

Answer: C (L2)

Q10

Box blur looks "boxy" because: A. the kernel has hard edges B. it's only 3×3 C. it's slow D. it doesn't normalise

Answer: A (L2)

Module 7 — Assignment: Adaptive Denoiser

Estimated time: 2 hours

Combines: Module 7 + Modules 2-6

Brief

Build a CLI that auto-picks the right denoising filter based on simple noise diagnostics.

Requirements

- `denoise.py INPUT [--output OUT]`
- Diagnose noise type:
- Count "extreme" pixels (value 0 or 255). If >0.5% of total → impulse noise.
- Estimate Gaussian noise σ by std of a small high-pass filtered region.
- Choose filter:
- Impulse-heavy → `cv2.medianBlur(img, 5)`.
- Mostly Gaussian → `cv2.bilateralFilter(img, 9, 75, 75)`.
- Low overall noise → `cv2.GaussianBlur(img, (3,3), 0)`.
- Print: diagnosis stats + chosen filter.
- Save result.

Constraints

- Modules 1-7 only.
- Use `argparse`.
- Diagnostic functions in their own namespaces.

Expected Output

```
$ python denoise.py noisy.png
extreme pixel %: 6.8
estimated gaussian sigma: 2.1
chosen filter: medianBlur(k=5) (heavy impulse noise)
saved -> noisy_denoised.png
```

Grading Rubric

Criterion	Weight
Diagnosis correct	30%
Right filter chosen	25%
Clear reporting	15%
Code organisation	15%
Argparse + style	15%

Submission

Save as `assignment_module7.py`.

Module 7 — Mini Project: Filter Playground (Trackbars)

Overview

An interactive playground with trackbars for all four smoothing filters: box, Gaussian, median, bilateral. Apply any of them in real time and compare against the original.

Concepts Used

- All Module 7 filters
- Trackbars (M2 L3)

Features

- One trackbar per param: filter type (0..3), kernel size, Gaussian σ , bilateral sigmaColor, bilateral sigmaSpace.
- Side-by-side display: original on the left, filtered on the right.
- s saves; q quits.

Starter Code

filter_playground.py:

```
import sys, cv2, numpy as np

FILTERS = ["box", "gaussian", "median", "bilateral"]

def apply_filter(img, name, k, sigma, sc, ss):
    k = max(1, k)
    if k % 2 == 0: k += 1
    if name == "box":
        return cv2.blur(img, (k, k))
    if name == "gaussian":
        return cv2.GaussianBlur(img, (k, k), sigma)
    if name == "median":
        return cv2.medianBlur(img, k)
    if name == "bilateral":
        return cv2.bilateralFilter(img, k, sc, ss)

def run(path):
    img = cv2.imread(path)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 700:
        img = cv2.resize(img, None, fx=0.5, fy=0.5)

    cv2.namedWindow("filters")
    cv2.createTrackbar("filter 0=box 1=gauss 2=med 3=bilat", "filters", 1, 3,
lambda v: None)
    cv2.createTrackbar("ksize", "filters", 5, 31, lambda v: None)
    cv2.createTrackbar("gauss sigma", "filters", 0, 10, lambda v: None)
    cv2.createTrackbar("bilat sColor", "filters", 75, 200, lambda v: None)
    cv2.createTrackbar("bilat sSpace", "filters", 75, 200, lambda v: None)
```

```

while True:
    f = FILTERS[cv2.getTrackbarPos("filter 0=box 1=gauss 2=med 3=bilat",
"filters")]
    k = cv2.getTrackbarPos("ksize", "filters") or 1
    sg = cv2.getTrackbarPos("gauss sigma", "filters") or 0
    sc = cv2.getTrackbarPos("bilat sColor", "filters")
    ss = cv2.getTrackbarPos("bilat sSpace", "filters")
    out = apply_filter(img, f, k, sg, sc, ss)
    combo = np.hstack([img, out])
    cv2.putText(combo, f"{f} k={k}", (10, 25),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.imshow("filters", combo)
    kk = cv2.waitKey(30) & 0xFF
    if kk == ord('q'): break
    if kk == ord('s'): cv2.imwrite("filtered.png", out); print("saved
filtered.png")
    cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coffee.png")

```

Expected Interaction

```

$ python filter_playground.py coffee.png
(window opens; original | filtered side-by-side)
toggle filter trackbar, change ksize, etc.
press s -> saved filtered.png
press q -> quit

```

Extension Challenges

- Add a "synthetic noise" toggle (Gaussian / salt-and-pepper) so you can stress-test filters.
- Add a PSNR / SSIM readout vs original.
- Add a "before / after" toggle key.

Grading Rubric

Criterion	Weight
All 4 filters work	35%
Trackbars responsive	20%
Side-by-side display	15%
Save / quit	10%
Style	20%